

# Plateforme de Gestion de Hackathon

Rapport de projet — Spring Boot & Angular

Application web complète de gestion d'un hackathon :  
inscription, équipes, projets, évaluation par jury et classement en temps réel

Durée du projet : 2 semaines  
Stack technique : Spring Boot 4 · Angular (standalone) · MySQL

# Sommaire

---

1. Contexte et objectifs
2. Architecture technique
3. Modèle de données
4. Sécurité et authentification
5. Fonctionnalités backend (API REST)
6. Frontend Angular
7. Fonctionnalités bonus
8. Acteurs et parcours utilisateurs
9. Difficultés rencontrées et solutions
10. Conclusion et perspectives

# 1. Contexte et objectifs

Les hackathons rassemblent des participants en équipes pour développer des projets innovants en un temps limité. Leur organisation nécessite des outils dédiés permettant l'inscription des participants, la gestion des équipes, la soumission des projets et leur évaluation par un jury.

L'objectif de ce projet est de fournir une application web complète répondant à ces besoins, avec une séparation claire des rôles (participant, jury, administrateur) et une sécurité robuste de bout en bout.

## Objectifs fonctionnels

- Permettre l'inscription et l'authentification sécurisée des utilisateurs
- Gérer la création et la composition des équipes
- Permettre la soumission et la modification des projets, avant une deadline configurable
- Permettre au jury de noter les projets selon trois critères (innovation, technique, présentation)
- Calculer et afficher un classement automatique des équipes
- Offrir à l'administrateur des outils de supervision (utilisateurs, équipes, projets, réglages)

# 2. Architecture technique

L'application suit une architecture **client-serveur découplée** : un frontend Angular communique avec un backend Spring Boot exposant une API REST, elle-même connectée à une base de données relationnelle MySQL.

## Backend — Spring Boot

| Composant                   | Rôle                                               |
|-----------------------------|----------------------------------------------------|
| Spring Web                  | Exposition de l'API REST (contrôleurs)             |
| Spring Security + JWT       | Authentification stateless et gestion des rôles    |
| Spring Data JPA / Hibernate | Persistance des données, mapping objet-relationnel |
| Spring WebSocket (STOMP)    | Diffusion du classement en temps réel              |
| springdoc-openapi (Swagger) | Documentation interactive de l'API                 |

## Frontend — Angular (standalone)

| Composant             | Rôle                                                        |
|-----------------------|-------------------------------------------------------------|
| Composants standalone | Un composant par fonctionnalité, sans NgModule              |
| Signals + RxResource  | Gestion réactive de l'état et des appels HTTP en lecture    |
| Route Guards          | Protection des routes par authentification et par rôle      |
| HTTP Interceptor      | Injection automatique du token JWT dans chaque requête      |
| Environments          | Séparation des configurations développement / production    |
| STOMP.js              | Connexion WebSocket au serveur pour le classement en direct |

## Base de données

MySQL, avec génération et mise à jour automatique du schéma via Hibernate (*ddl-auto=update*).

### 3. Modèle de données

---

Le schéma relationnel s'articule autour de six entités principales :

| Entité            | Description                                                                   |
|-------------------|-------------------------------------------------------------------------------|
| User              | Compte utilisateur : identifiants, e-mail, mot de passe haché, rôle           |
| Team              | Équipe participant au hackathon, avec un créateur                             |
| TeamMember        | Table d'association User ↔ Team (appartenance à une équipe)                   |
| Project           | Projet soumis par une équipe (titre, description, lien GitHub, fichier joint) |
| Evaluation        | Note attribuée par un jury à un projet (3 critères + commentaire)             |
| HackathonSettings | Réglages globaux (deadline de soumission)                                     |
| Notification      | Message adressé à un utilisateur suite à un événement                         |

#### Règles d'intégrité principales

- Un participant ne peut appartenir qu'à une seule équipe à la fois
- Une équipe ne peut soumettre qu'un seul projet
- Un jury ne peut noter un même projet qu'une seule fois (mise à jour possible)
- Une équipe est automatiquement supprimée si elle devient vide
- Un utilisateur membre d'une équipe ne peut être supprimé par l'administrateur sans en être retiré au préalable

## 4. Sécurité et authentification

---

L'authentification repose sur des **jetons JWT (JSON Web Token)**, garantissant une API totalement *stateless* : le serveur ne conserve aucune session, chaque requête porte sa propre preuve d'identité.

### Flux d'authentification

- L'utilisateur s'inscrit (mot de passe haché avec BCrypt, jamais stocké en clair)
- À la connexion, le serveur vérifie les identifiants et génère un JWT signé, contenant le nom d'utilisateur et le rôle
- Le client stocke ce jeton et le transmet dans l'en-tête *Authorization: Bearer <token>* à chaque requête suivante
- Un filtre serveur (JwtAuthFilter) intercepte chaque requête, valide le jeton et authentifie l'utilisateur auprès de Spring Security

### Gestion des rôles

Trois rôles distincts structurent les droits d'accès : **ROLE\_PARTICIPANT** (créer/rejoindre une équipe, soumettre un projet), **ROLE\_JURY** (évaluer les projets) et **ROLE\_ADMIN** (supervision globale). Les contrôles sont appliqués à deux niveaux : au niveau des routes (Spring Security) et au niveau des données (vérifications métier dans les services, par exemple pour garantir qu'un participant ne modifie que le projet de sa propre équipe).

### Gestion centralisée des erreurs

Un gestionnaire global d'exceptions (*GlobalExceptionHandler*) intercepte toutes les erreurs de l'application et les traduit en réponses JSON uniformes, avec un code HTTP cohérent (400, 401, 403, 409, 500), facilitant leur exploitation côté frontend.

## 5. Fonctionnalités backend (API REST)

### Authentification

| Méthode | Route              | Description                         |
|---------|--------------------|-------------------------------------|
| POST    | /api/auth/register | Inscription d'un nouvel utilisateur |
| POST    | /api/auth/login    | Connexion, retourne un token JWT    |

### Équipes

| Méthode | Route                 | Description          |
|---------|-----------------------|----------------------|
| POST    | /api/teams            | Créer une équipe     |
| POST    | /api/teams/{id}/join  | Rejoindre une équipe |
| POST    | /api/teams/{id}/leave | Quitter une équipe   |
| GET     | /api/teams            | Lister les équipes   |

### Projets

| Méthode | Route                   | Description                         |
|---------|-------------------------|-------------------------------------|
| POST    | /api/projects           | Soumettre un projet                 |
| PUT     | /api/projects/{id}      | Modifier un projet (avant deadline) |
| GET     | /api/projects           | Lister les projets                  |
| POST    | /api/projects/{id}/file | Joindre un fichier au projet        |
| GET     | /api/projects/{id}/file | Télécharger le fichier joint        |
| DELETE  | /api/projects/{id}/file | Supprimer le fichier joint          |

### Évaluation et classement

| Méthode | Route                             | Description                                     |
|---------|-----------------------------------|-------------------------------------------------|
| POST    | /api/jury/evaluations/{projectId} | Noter (ou mettre à jour la note d'un) un projet |
| GET     | /api/jury/projects                | Lister les projets à évaluer                    |
| GET     | /api/jury/evaluations/mine        | Évaluations réalisées par le jury connecté      |
| GET     | /api/leaderboard                  | Classement des équipes                          |

### Administration

| Méthode | Route                        | Description                       |
|---------|------------------------------|-----------------------------------|
| GET     | /api/admin/users             | Lister les utilisateurs           |
| PUT     | /api/admin/users/{id}/role   | Modifier le rôle d'un utilisateur |
| DELETE  | /api/admin/users/{id}        | Supprimer un utilisateur          |
| DELETE  | /api/admin/teams/{id}        | Supprimer une équipe              |
| DELETE  | /api/admin/projects/{id}     | Supprimer un projet               |
| PUT     | /api/admin/settings/deadline | Définir la deadline de soumission |

*L'ensemble de l'API est documenté et testable via Swagger (springdoc-openapi), accessible à l'adresse /swagger-ui/index.html.*

## 6. Frontend Angular

---

Le frontend est développé en Angular moderne (composants *standalone*, sans NgModule), avec une identité visuelle cohérente : une barre latérale de navigation adaptée au rôle connecté, et une palette bleu marine / doré.

### Pages principales

- **Connexion / Inscription** — formulaires avec validation et retour d'erreur du serveur
- **Équipes** — création, adhésion, départ, visualisation des membres
- **Projets** — vue adaptée au rôle : formulaire de soumission pour le participant, bouton d'évaluation pour le jury, suppression pour l'administrateur
- **Classement** — affichage du podium, mis à jour en temps réel via WebSocket
- **Administration** — gestion des utilisateurs et de la deadline de soumission

### RxResource — gestion réactive des données

Les listes de données (équipes, projets, classement) sont chargées via l'API *RxResource*, qui encapsule automatiquement l'état de chargement, la donnée et les erreurs sous forme de signaux réactifs, sans code de gestion manuel.

### Route Guards

Deux gardes protègent la navigation : un garde d'authentification (redirige vers la connexion si l'utilisateur n'est pas identifié) et un garde de rôle, paramétrable, qui restreint l'accès de certaines pages à un rôle précis.

### Intercepteur HTTP JWT

Un intercepteur fonctionnel ajoute automatiquement l'en-tête d'autorisation à chaque requête sortante disposant d'un token, sans intervention manuelle dans les services métier.

### Environnements

Deux fichiers de configuration distincts (développement / production) définissent l'URL de l'API selon le contexte d'exécution, sans jamais coder cette valeur en dur dans le code source.

## 7. Fonctionnalités bonus

---

Au-delà des exigences obligatoires du cahier des charges, les fonctionnalités bonus suivantes ont été implémentées :

| Bonus                    | Description                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------|
| Deadline de soumission   | Réglage global configurable par l'administrateur ; la soumission et la modification de projet sont |
| Badge gagnant            | L'équipe en tête du classement est mise en évidence (badge doré) sur le classement et sur la lis   |
| Notifications            | Un système de notifications informe les membres d'une équipe lorsqu'un coéquipier rejoint ou q     |
| Upload de fichier projet | Une équipe peut joindre un fichier (PDF, archive...) à son projet, en complément du lien GitHub,   |
| Temps réel (WebSocket)   | Le classement se met à jour instantanément chez tous les utilisateurs connectés dès qu'une nou     |

## 8. Acteurs et parcours utilisateurs

---

### Participant

- S'inscrit et se connecte
- Crée ou rejoint une équipe (une seule à la fois)
- Soumet le projet de son équipe et peut le modifier jusqu'à la deadline
- Peut joindre un fichier à son projet
- Consulte le classement en direct

### Jury

- Consulte l'ensemble des projets soumis
- Attribue une note selon trois critères : innovation, technique, présentation
- Peut modifier une note déjà attribuée
- Visualise, via un badge, les projets déjà évalués

### Administrateur

- Consulte et gère les comptes utilisateurs (changement de rôle, suppression contrôlée)
- Supprime des équipes ou des projets si nécessaire
- Définit la deadline de soumission des projets

## 9. Difficultés rencontrées et solutions

| Difficulté                                                           | Solution retenue                                                                                      |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Incompatibilités de version (Spring Boot 4 / Hibernate 6+)           | Mise à jour forcée des dépendances concernées vers des versions compatibles                           |
| Génération automatique du dialecte Hibernate obsolète (MySQLDialect) | Suppression du dialecte et configuration explicite : Hibernate 6+ détecte le dialecte automatiquement |
| Conflit d'intérêt possible (un jury notant son propre projet)        | Réstriction de la création/adhésion à une équipe au seul rôle participant, conformément aux règles    |
| Incompatibilité de sockjs-client avec l'environnement de base        | Basculer vers une version globale de SockJS (tentative avortée)                                       |
| Cohérence des messages d'erreur affichés côté client                 | Mise en place d'un gestionnaire d'exceptions global côté serveur, renvoyant des messages cohérents    |

## 10. Conclusion et perspectives

---

Ce projet a permis de mettre en œuvre une application full-stack complète, couvrant l'ensemble des exigences du cahier des charges : authentification sécurisée par JWT, gestion des rôles, création et administration d'équipes et de projets, évaluation structurée par un jury et calcul automatique d'un classement. L'architecture retenue (API REST découplée, frontend réactif à base de signaux) offre une base solide, évolutive et maintenable.

Au-delà du socle obligatoire, cinq fonctionnalités bonus ont été ajoutées : la gestion d'une deadline de soumission, un badge de mise en avant de l'équipe gagnante, un système de notifications, la possibilité de joindre un fichier au projet, et surtout un classement diffusé en temps réel via WebSocket — une fonctionnalité avancée démontrant la capacité du système à aller au-delà d'un simple échange requête/réponse.

### Pistes d'amélioration possibles

- Transfert automatique de la propriété d'une équipe lorsque son créateur la quitte
- Historique et export du classement final une fois le hackathon terminé
- Notifications en temps réel (et non plus seulement consultées au rafraîchissement)
- Tests automatisés (unitaires et d'intégration) sur les règles métier critiques
- Interface d'administration enrichie (statistiques globales, export CSV)

---

*Rapport généré dans le cadre du projet « Plateforme de gestion de Hackathon » — Spring Boot & Angular.*